Patrick Sheehy

2026-03-08

Foundations of Python

Assignment07

Link to Git repo

Class Inheritance: Standing on the Shoulders of Giants!

Or don't repeat work you have already done!

Intro

This module has us starting to build object classes to help us customize how we want data handled in our programs. This also means learning how to create and document the classes functions (called methods) and internal data (called attributes). We also touch on class inheritance and how that can be used to make parent-child classes and simplify the amount of work needed as the child class can pull from methods and attributes of the parent. Finally, the GitHub Desktop app and GIT functionality in PyCharm were introduced as alternative workflows to using the web interface.

Stretching My Skills

I don't have much previous experience with Object Oriented Programing, so this week has started to push me to spend more time examining the labs/demos/videos to bring my understanding and skills up to what is needed for the assignment. This led to my first real discussion post to clarify my misunderstanding of the reading and requirements for Lab02. I hadn't fully understood that, for now, when we make the attributes of a class private, we cannot do this in the initial constructor without also having to repeat the validation checks. All this has meant that I am now building on both my previous programming knowledge and what we have learned in this class, and it is fun to think about how this knowledge expands my abilities. It is also fun to look back at my early assignments to see how far I've come, as we talked about in the QA session.

Understanding Class Potential

Up until now I hadn't really understood the full power that custom class objects can bring. For example, we can group functions into classes to "clean up" the main program and bring order to all the functions, but the real prize is being able to define a variable that holds what I want, how I want it held, and then prints what I decide! I like the fact that our current understanding of classes mans we can tell the computer how we want our custom class to appear in print statements, and I predict other ways in the future; like making our objects compatible with the JSON format by default. It also is another process that helps to "simplify" code to make it more readable and clearer. Figure 1 shows how a custom class can override the class string (what the print statement sees), and then Figure 2 shows how this simplifies outputting our table of student data. The actual code that is occurring here is much less complex than the commented-out code implies, especially for anyone that doesn't care how student is being displayed and just wants to understand what is occurring at this stage of the script.

Figure 1: String Override for custom class example

```
def __str__(self):
    """
    Return method for Student Class object

    ChangeLog: (Who, When, What)
        PSheehy, 2026-03-06, Created Method

    :return: How Student should be printed.
    """
    return f"{self.first_name}, {self.last_name}, {self.course_name}"
```

Figure 2: Custom class simplification of code

```
for student in student_data:

    # TODO Add code to access Student object data instead of dictionary data (DONE)
    #print(f'Student {student["FirstName"]} '
    #      f'{student["LastName"]} is enrolled in {student["CourseName"]}')
    print(student)
```

Testing the Script

Like last week, none of the printed "output" of the script has changed very much because that isn't the focus of this module. However, I did not pay enough attention to the requirements last week and missed the fact that the data written to a file should be displayed. This week I will still minimize the output screen captures I'm sharing, but will also pay more attention to the listed requirements for the program function. Figure 3 shows how the program takes new input and stores the data into the json file Enrollments of Figure 4.

Figure 3: PyCharm and Cmd Proof

```
Enter the student's first name: Patrick
Enter the student's last name: Sheehy
Enter the name of the registered course: ENGR 400

You have registered Patrick Sheehy for Engr 400.


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 3
-------------------------------------------------
Vic, Vu, Python 100
Sue, Jones, Python 100
Patrick, Sheehy, Engr 400
-------------------------------------------------
Student data saved to file
```

```
Enter the student's first name: Tim
Enter the student's last name: smith
Enter the name of the registered course: phys 200

You have registered Tim Smith for Phys 200.


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
----------------------------------------


Enter your menu choice number: 3
-------------------------------------------------
Vic, Vu, Python 100
Sue, Jones, Python 100
Patrick, Sheehy, Engr 400
Tim, Smith, Phys 200
-------------------------------------------------
Student data saved to file
```

Figure 4: Enrollments final state

Assignment07.py | Enrollments.json


```
[
  {
    "FirstName": "Vic",
    "LastName": "Vu",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Sue",
    "LastName": "Jones",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Patrick",
    "LastName": "Sheehy",
    "CourseName": "Engr 400"
  },
  {
    "FirstName": "Tim",
    "LastName": "Smith",
    "CourseName": "Phys 200"
  }
]
```

Summary

The tools we are learning to use this week (Custom Class objects, Inheritance, and Git interfaces) are all about improving our script's readability as well as the ease of which we can breakdown complex problems into simple scripts. It is amazing how far we have come in only 7 weeks, and also a little sad that we are nearing the end of the course…

Table of Figures

Figure 1: String Override for custom class example 2
Figure 2: Custom class simplification of code 2
Figure 3: PyCharm and Cmd Proof 3
Figure 4: Enrollments final state 3